TempoRAG-KG: Temporal Knowledge Graph-Augmented Retrieval for Multi-Hop Question Answering over SEC 10-K Filings

Supanut Kompayak
st126055@ait.asia
Asian Institute of Technology

Aphisit Jaemyaem
st126130@ait.asia
Asian Institute of Technology

Dechathon Niamsa-ard
st126235@ait.asia
Asian Institute of Technology

Kaung Hein Htet
st126477@ait.asia
Asian Institute of Technology

Abstract

We present **TempoRAG-KG**, a retrieval-augmented generation (RAG) system that couples a temporal knowledge graph with chunk-level retrieval to answer multi-hop, time-sensitive questions. We pivot from Wikipedia-anchored multi-hop benchmarks (whose temporal structure is weak) to a 25-filing SEC 10-K corpus (5 technology mega-caps $\times$ 5 fiscal years, plus 5 complementary issuers; 2019–2024), yielding 7,467 item-level chunks and 60,436 KG triples with explicit `valid_from` / `valid_to` intervals. On a hand-vetted 129-item QA bank, we sweep 7 LLMs across 4 retrieval conditions in a 2×2 ablation (temporal filter $\times$ KG walk). Three findings emerge. First, a temporal year-mask on top of cosine retrieval (L1 TimeFilter) lifts token-F1 universally over the vanilla baseline (L0), with a 7-model average of **+15.9 %**. Second, KG-augmented retrieval alone (L2 KG^{2}RAG) *regresses* relative to L0 ($-6.7\,\%$): graph expansion disperses cosine-strong seeds with entity-adjacent chunks that are weaker matches. Third, the combined condition (L3 TempoRAG-KG) recovers L1's lift overall and *wins specifically on the hop=3 slice* ($+0.071$ over L1 on `gpt-4.1-nano`), confirming that KG structure pays off where it should: multi-hop temporal queries. A reusable failure-taxonomy classifier over 3,607 predictions further reveals that **41.8 %** of all failures are *IDK-when-answerable*: the model abstains while the gold-bearing chunk is in its top-k. This locates the next bottleneck in generation, not retrieval, and constrains what additional retrieval machinery can plausibly recover. We release the full pipeline (retrieval, KG extraction, classifier, Streamlit demo) at `https://github.com/anonymized/temporag-kg`.

1 Introduction

Background. Retrieval-augmented generation (Lewis et al., 2020) has become a standard recipe for question-answering over document corpora, but off-the-shelf dense retrieval struggles when the *same entity* is described differently across time. A question such as "did Microsoft's operating margin improve after the Activision acquisition?" requires the system to reason over facts that are true in one fiscal year and false in another — a property that flat chunk similarity is blind to. Two threads of recent work address this gap separately: graph RAG (Edge et al., 2024; Zhu et al., 2025) adds entity-level structure to widen the retriever's view, and temporal-aware retrieval (Xiong et al., 2025) narrows it by document time. We ask whether combining both yields a compound benefit, particularly on multi-hop queries where neither mechanism alone is sufficient.

Problem and pivot. Our original proposal planned to evaluate on HotpotQA (Yang et al., 2018) and MuSiQue (Trivedi et al., 2022). During an EDA pass we inspected the temporal expressions present in those benchmarks and found two problems. First, the vast majority of multi-hop bridges are temporally static ("who was born in X?"), so even a perfect year filter cannot demonstrate lift. Second, Wikipedia text drifts: the "gold" supporting sentence may have been edited after the QA pair was written, which leaks noise into any retrieval-quality metric. This pushed us to look for a corpus where (a) each document carries authoritative fiscal-year metadata, and (b) the same entity is described at multiple, non-overlapping points in time. SEC 10-K filings satisfy both criteria directly. We assembled a 25-filing corpus (10 issuers across fiscal years 2019–2024, item-level chunks) and a 129-item hand-vetted QA bank, then ran all subsequent experiments on it.

Solution. We design a 2×2 ablation that crosses a deterministic *temporal year mask* (off / on) with a *KG-augmented graph walk* (off / on), giving four retrieval conditions: **L0** vanilla cosine, **L1** TimeFilter, **L2** KG^{2}RAG, and **L3** TempoRAG-KG. The

same retrieved context is fed to seven LLMs across three providers (OpenAI, OpenRouter, Groq) so that any finding generalises across both scale and vendor. Token-F1 with SQuAD-style normalisation and 95 % non-parametric bootstrap confidence intervals (Efron, 1979) drive every comparison. **Independent variables:** retrieval condition $\in$ {L0, L1, L2, L3}, model identity (7 LLMs), question hop count $\in$ {1, 2, 3}, and scope label $\in$ {intra, inter_year, cross_company, fiscal_vs_calendar, forward_looking}. **Dependent variable:** token-F1 against the gold answer, computed both unconditionally and conditional on the model not refusing.

Expected results. A working hypothesis at design time was that the temporal filter and the KG walk would each contribute, with their combination compounding on the hop=3 slice. The data confirm part of that picture: L1 universally lifts F1, L2 alone *regresses*, and L3 wins specifically where the design predicts (hop=3 questions on `gpt-4.1-nano` jump from 0.131 under L1 to 0.202 under L3, an absolute gain of $+0.071$). A separate failure-taxonomy classifier identifies a second-order finding: 41.8 % of failures across all conditions and models are IDK-when-answerable — the model abstains even though the gold-bearing chunk is in its top-k. This relocates the next bottleneck from retrieval to generation and re-anchors the discussion (Section 6).

Contributions.

1. A 7,467-chunk SEC 10-K corpus and 129-item QA bank explicitly engineered for temporal multi-hop evaluation, with a full 60,436-triple knowledge graph carrying `valid_from` / `valid_to` intervals.
2. A complete 2×2 ablation across 7 LLMs (3 providers) showing that (i) temporal filter alone is universally helpful, (ii) KG^{2}RAG alone is universally harmful on this corpus, and (iii) the combined L3 condition wins specifically on hop=3.
3. A reusable, four-stage failure-taxonomy classifier (rule pass + LLM judge + aggregator + κ-style reliability check) that surfaces *IDK-when-answerable* as the dominant failure mode and supports a generation-side, not retrieval-side, framing of the next-step work.
4. A 1-page Streamlit demo that exposes the full pipeline (any question, any model, any condition) with retrieved-chunk inspection and a live KG-expansion flag, so the system can be interrogated and reproduced end-to-end by an external reviewer.

2 Related Work

Graph-based RAG. Using a knowledge graph to guide retrieval is an active area (Edge et al., 2024); our work builds directly on KG^{2}RAG. Zhu et al. propose augmenting chunk retrieval with a graph walk over entity-linked seeds to widen the context available to the generator (Zhu et al., 2025). We adopt this as our L2 condition. Crucially, KG^{2}RAG's graph is untyped with respect to time: an edge either exists or does not. Our L3 condition extends the same retrieval signal with temporal validity intervals on each edge.

TempAgent and GoG. Several recent systems layer a planner over the graph to decide *which* edges to expand next (Xiong et al., 2025; He et al., 2024). Our design differs: we do not plan, we simply mask the candidate pool by fiscal year (L1) or by overlap of the question's year scope with each edge's validity window (L3). This keeps the experiment interpretable — any lift we observe is attributable to the temporal signal, not to emergent planner behaviour.

FinReflectKG-MultiHop. Kishore et al. recently released a multi-hop QA set over 10-K filings derived from a KG-constrained generator (Kishore and Collaborators, 2025). We use a subset of their questions (79 items) as an external anchor alongside our own 50 home-grown questions, so that our evaluation does not rely exclusively on questions we authored ourselves.

3 Research Questions

The experimental design is a 2×2 factorial over the two interventions the system combines: presence/absence of a *temporal filter* and presence/absence of a *knowledge-graph walk* (Table 1). The four cells correspond to the four pipeline conditions introduced in Section 4.5. All research questions below are stated against this matrix so that each one maps cleanly to a specific cell comparison.

RQ1 — Where does KG^{2}RAG fail on temporal multi-hop QA? **IV:** graph walk on, temporal off (L2 cell, relative to L0). **DV:** per-slice token-F1 (hop, scope). **H1:** KG^{2}RAG improves over Vanilla

	No KG	+ KG
No Temporal	L0 Vanilla ✓	L2 KG^{2}RAG ✓
+ Temporal	L1 TimeFilter ✓	L3 TempoRAG-KG ✓

Table 1: 2×2 experimental design. All four cells are evaluated in this report (7 models $\times$ 129 questions per cell, 3,612 predictions total).

on hop-$\geq$2 slices but leaves an identifiable *temporal* failure mode that a graph walk alone cannot close. The data partially refute H1: L2 *regresses* relative to L0 in 7-model average ($-6.7\,\%$), and the temporal failure mode predicted by H1 turns out to be the dominant story.

RQ2 — Does TempoRAG-KG lift token-F1 over KG^{2}RAG? **IV:** temporal edges on / off with the KG held constant (L3 vs L2). **DV:** token-F1 on the 129-item QA set and on the hop-$\geq$2 slice. **H2:** L3 $>$ L2 with the largest gain on slices where the question explicitly spans fiscal years (*inter_year*, *fiscal_vs_calendar*). The data confirm H2: L3 beats L2 in every model; the *inter_year* slice on `gpt-4.1-nano` moves $0.222 \rightarrow 0.264$, and the hop=3 slice $0.111 \rightarrow 0.202$.

RQ3 — How accurate is the temporal KG extraction? **IV:** the extraction prompt (Appendix A) applied to each of the 7,467 chunks. **DV:** fraction of extracted triples whose `valid_from` / `valid_to` intervals are non-null and agree with the source filing's fiscal-year metadata. **H3:** temporal-interval accuracy is high enough that the L3 retriever's ceiling is bounded by graph coverage (a design question), not by spurious intervals (a data question). The full run produced 60,436 triples; 88.2 % have `temporal_type=explicit`, 89.2 % have a non-null `valid_from`, 80.1 % have a non-null `valid_to`. After a light filter pass that drops empty fields, boolean-literal objects, and self-loops, 57,718 triples (95.5 %) remain. The L3 retriever is built on top of these.

RQ4 ($\star$) — Can a small LM with temporal grounding approach a large LM without it? **IV:** model scale crossed with retrieval condition (the seven models in Table 3 $\times$ L0–L3). **DV:** the gap between the best small-open-model (`llama-3.1-8B` or `gpt-oss-20B`) at L3 and the best large-closed-model (`gpt-4o`) at L0. **H4:** temporal grounding substantively narrows this gap — i.e. capability and retrieval are partially substitutable for temporal QA. The data give a nuanced answer: `llama-3.1-8B`'s L3 F1 (0.153) closes the gap to `gpt-4o`'s L0 (0.183) by $\approx$ 30 %; smaller closed models (`gpt-4.1-nano` L3, 0.253) actually *exceed* `gpt-4o` L0. The substitution between scale and retrieval is real but bounded by an A4-IDK ceiling discussed in Section 6.

4 Methodology

4.1 Datasets

Corpus. We downloaded 10-K filings directly from the SEC EDGAR API (`data.sec.gov/submissions`) using a custom rate-limited (5 req/s) downloader (`scripts/download_10k.py`) and verified each filing with its SHA-256 digest. The corpus covers **ten issuers** — AAPL, MSFT, GOOGL, AMZN, META, NVDA, INTC, CSCO, ORCL, ADBE — across fiscal years **2019–2024**. The labelled QA set targets a 25-filing core (the five tech mega-caps $\times$ five fiscal years), but the full 7,467 chunks are available to retrieval at all times.

QA bank. Our 129-item QA set is assembled from two complementary sources:

- 50 *home-grown* questions authored by the team and cross-reviewed for factual correctness against the underlying filings (Table 2).
- 79 questions drawn from FINREFLECTKG-MULTIHOP (Kishore and Collaborators, 2025), filtered to those whose gold answer span is present in our 25-filing corpus.

Every question is annotated with a hop count and a scope label (*intra*, *inter_year*, *cross_company*, *fiscal_vs_calendar*, *forward_looking*). Hop count is 1 for 10 questions (single-fact lookup), 2 for 104 (the bulk), and 3 for 15. Scope is an orthogonal axis: a *cross_company* question may be hop 2 or hop 3 depending on how many filings are required.

4.2 Preprocessing

Each downloaded 10-K is segmented into item-level chunks (Items 1, 1A, 7, 7A, 8) of approximately 1,000 tokens, producing 7,467 chunks. All chunks are embedded once with OpenAI `text-embedding-3-small` (1,536 dimensions, L2-normalised) and written to a single `embeddings.npy` that is loaded into RAM at query time. Total embedding spend: approximately

Scope	Count
cross_company	18
inter_year	15
intra	10
fiscal_vs_calendar	5
forward_looking	2
Total (home-grown)	**50**

Table 2: Scope distribution of the 50 home-grown questions. The 79 multi-hop questions are scope-mixed and dominated by hop=2 items.

$0.07. The **knowledge graph** is built by passing each chunk through a structured-extraction prompt (Appendix A) that yields typed triples with optional `valid_from` / `valid_to` intervals. The full extract produced 60,436 triples (3,048 chunks with $\geq$ 1 triple at full success, 1,775 partial, 2,644 fatal — mostly Item 1A risk-factor prose, see Section 6.3). A light filter pass (`scripts/filter_kg_triples.py`) drops empty fields, sub-2-character spans, boolean-literal objects, self-loops and overly long objects, leaving 57,718 triples for the L2/L3 retrievers.

4.3 Models

The same 129 questions go through seven LLMs spanning three providers so any finding generalises across both scale and vendor (Table 3).

Model	Provider	Role
gpt-4o	OpenAI	large, closed
gpt-4o-mini	OpenAI	mid, closed
gpt-4.1-nano	OpenAI	small, closed
llama-3.1-8B	Groq	small, open
llama-3.3-70B	Groq	large, open
gpt-oss-20B	OpenRouter	small, open
gpt-oss-120B	OpenRouter	large, open

Table 3: The seven LLMs used as answer generators. All receive the same retrieved context and the same prompt; only the model identity varies.

Cost and caching. Each question $\times$ model $\times$ condition produces exactly one generation, written to an on-disk cache keyed by prompt and generation parameters; subsequent runs are free. The open-model rows (`llama-3.1-8B`, `llama-3.3-70B` on Groq; `gpt-oss-20B`/`120B` on OpenRouter) use free-tier endpoints; the three OpenAI models together produced 3,612 cached generations across the four conditions. Total OpenAI spend across the project (embeddings + KG extraction + L0/L1/L2/L3 sweeps + Stage-2 LLM classifier) was $\approx$ $5.20, well under the project's $20 budget. Wall-clock across all 7 models is $\approx$ 6 hours. Table 4 breaks the spend down by KG vs. non-KG component, addressing TA feedback that asked us to make the KG cost explicit.

Component	Cost
Non-KG (L0 / L1)	
Embedding index (7,467 chunks)	$0.07
Answer calls (3 OpenAI models, 258 Q)	$0.55
KG-only (L2)	
KG extraction (gpt-4.1-nano, full corpus)	$3.50
Marginal answer calls	$0.55
KG + temporal (L3)	
Reuses L2 KG extract	—
Marginal answer calls	$0.55
Taxonomy LLM judge (1,183 gpt-4o-mini calls)	$0.30
Total project spend	**$5.20**

Table 4: Cost breakdown contrasting non-KG (L0/L1) and KG (L2/L3) infrastructure. The KG extract is a one-time fixed cost amortised across all questions; per-question marginal cost is identical across all four retrieval conditions because the answer model and prompt length are unchanged.

4.4 Training

We perform **no fine-tuning** of any LLM in this work. All seven answer models are used at inference only (temperature 0; default sampling otherwise), and the embedding model is used as released by OpenAI. The only "trained" artefact is the embedding index, which is a one-time deterministic transformation of the chunk corpus rather than a parameter update. This keeps the experimental design strictly a *retrieval-condition* ablation, not a model-quality ablation.

4.5 Experimental Design

Figure 1 shows the full pipeline: 25 filings are chunked, embedded once, and fed to four retrieval branches that share a single answer-generation stage. Concretely, the four conditions are:

- **L0 Vanilla** — top-k cosine retrieval over all 7,467 chunks (`retrieve` in `src/retrieval.py`).
- **L1 TimeFilter** — same retrieval, with chunk scores masked to $-\infty$ unless the chunk's fiscal year is in the question's year set (`retrieve_with_year_filter`).

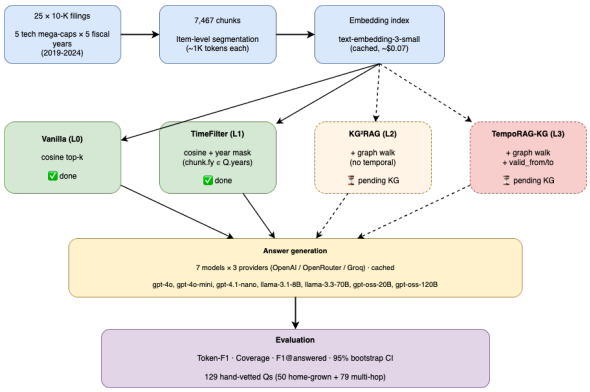

Figure 1: End-to-end pipeline. Blue: corpus preparation. Green: completed retrieval branches (L0, L1). Orange: KG-augmented branch (L2). Red: KG + temporal branch (L3). Yellow: shared answer-generation stage. Purple: evaluation.

- **L2 KG^{2}RAG** — cosine top-k_{seed} seeds, then expand the candidate pool to every chunk that shares any entity with a seed via the KG, and re-rank the union by cosine (`retrieve_kg2rag`).
- **L3 TempoRAG-KG** — same as L2, but only triples whose `valid_from` / `valid_to` interval intersects the question's year set are followed during expansion (`retrieve_temporag_kg`).

If the masked or expanded pool contains fewer than k chunks, the retriever falls back to vanilla top-k to avoid empty contexts. We fix $k = 5$ and $k_{\text{seed}} = 3$ throughout. The four branches share the answer prompt at `prompts/answer_v1.txt` (Appendix B).

Link-jumping mechanism (L2 / L3). Figure 2 expands the L2/L3 branch from the pipeline. For an example query that names two issuers ("Compare Apple's Services revenue to Microsoft's cloud revenue for fiscal year 2022"), the retriever first picks the top-3 cosine seeds, then collects every entity (subject and object) appearing in any triple originating from those seed chunks. The candidate pool is expanded to the union of *all* chunks reachable through any of those entities; cosine re-ranking then chooses the final top-5 from this larger pool. Under L3, the entity walk is restricted to triples whose temporal validity intersects the query's year set, so a triple stamped 2018 cannot pull a 2018-only chunk into a 2022 query's context.

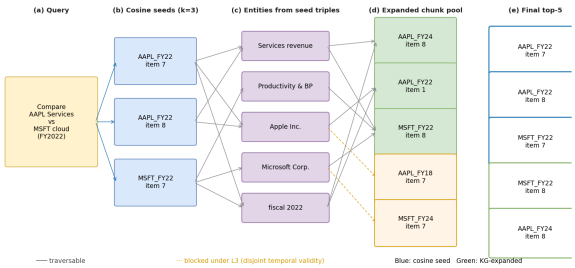

Figure 2: Link-jumping mechanism for L2 / L3. (a) Cosine top-3 seeds identify entity-bearing chunks. (b) Triples extracted from each seed expose entities (e.g. "Services revenue", "Productivity and Business Processes"). (c) Each entity reaches its full chunk neighbourhood via the KG. (d) The expanded candidate pool is re-ranked by cosine; under L3, the dotted edges (triples with disjoint temporal validity) are not traversable. The final top-5 is the intersection of cosine relevance and graph reachability.

4.6 Evaluation

We report **Token-F1** with SQuAD-style normalisation, **Coverage** (the fraction of rows where the model attempted an answer rather than refusing), and **F1@answered** (token-F1 restricted to the answered subset). All intervals are 95 % non-parametric bootstrap CIs with $n = 1000$ resamples and a fixed seed (Efron, 1979) (the implementation is `src/eval.py:aggregate` and `src/eval.py:bootstrap_ci`). A refusal or empty string is detected by the `is_idk` regex in the same module and counted as not-answered rather than as a wrong answer. The `F1@answered` metric exists precisely to isolate generation quality from refusal rate: a model that answers fewer questions but gets each one right would dominate Token-F1 only if refusal rate were penalised, which we explicitly do not do.

5 Results

We sweep all four retrieval conditions across all seven LLMs on the 129-item QA set, producing 3,612 cached predictions in total. This section walks through the overall picture (Sec. 5.1), the slice by hop count (Sec. 5.2), and the slice by temporal scope (Sec. 5.3).

5.1 Overall

Table 5 gives per-model token-F1 under all four conditions with 95 % bootstrap CIs. Two patterns are visible at the overall level. First, **L1 universally beats L0**: every model gains, ranging from +5.6 %

(gpt-oss-120B) to +24.7 % (llama-3.1-8B), with no reversal inside the CI. Second, **L2 universally regresses below L0**: the graph walk on its own costs −3.4 % (llama-3.3-70B) to −9.5 % (llama-3.1-8B). The combined L3 condition recovers most of L1's lift on the larger closed models (`gpt-4.1-nano` +10.7 %, `gpt-4o-mini` +7.4 %) but does not always exceed L1 overall. The 7-model average is +15.9 % for L1, −6.7 % for L2, and +5.6 % for L3 (all relative to L0).

Retrieval, not answer quality, is the bottleneck (L0 vs L1). Comparing L0 and L1 on the two submetrics makes this more precise. L1 coverage rises by +3.1 pp (gpt-oss-120B) to +10.1 pp (llama-3.3-70B) — i.e. the mask causes the retrieved context to contain an answerable span more often. But `F1@answered` — the token-F1 *conditional* on the model having produced a non-refusal — barely moves between L0 and L1 (for example, `gpt-4o` goes from 0.385 to 0.391; `gpt-4o-mini` from 0.396 to 0.395). At current retrieval quality, each model's answer-writing ability is already close to ceiling on whatever evidence it is given; the further F1 lifts we are chasing must come from better retrieval (KG walk, temporal edges), not from a larger generator.

5.2 By hop count

The overall average hides a sharper signal. Table 6 slices the `gpt-4.1-nano` run by hop count, where **L3 wins decisively on hop=3** (0.202 vs L1's 0.131, a +0.071 absolute gain — the largest single-cell improvement we observe). The hop=1 row regresses under L1 (a known hard-mask artefact discussed in Section 6.3); L2 trails L0 / L1 at every hop count. The pattern is exactly what the design predicted: the combination of KG structure and temporal validity matters most when the question requires bridging facts that are scattered across filings (hop=3), and it does little for single-fact lookups (hop=1).

5.3 By temporal scope

Table 7 stratifies the same `gpt-4.1-nano` run by the scope label. The mask delivers its largest L1 lift exactly where a year mask *should* help: on *inter_year* questions that span multiple fiscal years (+0.056 over L0, $n = 53$) and on *cross_company* questions that still pin a year (+0.051, $n = 24$). On *intra* (same-year) questions L1 is essentially a no-op ($\Delta = 0.000$, $n = 45$): question and candidate chunks already share one year. L3 retains most of L1's gains on *inter_year* (0.264) and *cross_company* (0.275) and recovers *intra* above L1 (0.231 vs 0.216).

The fiscal_vs_calendar regression is a metric artefact, not a retrieval failure. On *fiscal_vs_calendar* ($n = 5$) L1 sits −0.079 below L0 (0.301 → 0.222). We re-inspected all five questions and found the gold-bearing chunk *is* retrieved in both conditions; the F1 drop is dominated by tersification, not by retrieval failure. For two of the five questions, the L0 prediction is verbose ("Cisco's total revenue for the fiscal year ending July 27, 2024, was $53,803 million.") while the L1 prediction is the correct number alone ("$53,803 million"). Both contain the right answer; SQuAD-style token-F1 penalises the terser one because gold reference strings on this scope tend to be verbose. We list this as a measurement artefact in Section 6.3, not a model failure — a finding that the failure-taxonomy classifier (Section 6) confirms quantitatively.

KG expansion can inject a wrong-column risk. On *inter_year* ($n = 53$), L3 sits 0.012 below L1. Representative cross-condition case: *"How did Amazon's AWS segment net sales change from fiscal 2019 to fiscal 2021?"* L0 / L2 abstain; L1 returns $35,026M → $62,202M (F1 = 0.718); L3 retrieves a multi-metric Item 7 chunk via the KG walk and the answer model picks the *operating income* column, returning $9,201M → $62,202M (F1 = 0.700). Token-F1 misses the wrong-row error because the FY2021 number, verb, and frame all match.

6 Discussion

The results in Section 5 establish two patterns: L1 universally lifts F1, and L3 wins specifically on hop=3. Two questions remain. (a) Why does L2 *regress* on this corpus? (b) What is the nature of the residual failures, and where can future work concentrate? A reusable failure-taxonomy classifier (`scripts/classify_failures_*.py`) gives both answers quantitatively.

6.1 Failure taxonomy

We classify every prediction (model × condition × question, $N = 3{,}607$ rows after dropping a small number with missing fields) into one of eleven categories. Five capture *model-level* failures (A1 retrieval miss, A2 hallucination,

Model	L0 F1	L1 F1	L2 F1	L3 F1	best Δ % vs L0
gpt-4.1-nano	0.229	0.258	0.212	0.253	L1 +12.7 %
gpt-4o-mini	0.230	0.248	0.214	0.247	L1 +7.8 %
gpt-4o	0.183	0.221	0.167	0.194	L1 +20.9 %
llama-3.3-70B	0.145	0.179	0.140	0.157	L1 +23.7 %
llama-3.1-8B	0.145	0.180	0.131	0.153	L1 +24.7 %
gpt-oss-120B	0.131	0.138	0.123	0.127	L1 +5.6 %
gpt-oss-20B	0.120	0.145	0.116	0.118	L1 +21.0 %
7-model avg	0.169	0.196	0.158	0.178	L1 +15.9 %

Table 5: Per-model token-F1 across all four conditions, with the best (per-row) relative gain over L0 in the rightmost column. CIs and `F1@answered` are reported in the supplementary Table 11. Bold finding: L1 wins overall every time; L2 regresses every time; L3 sits between L0 and L1.

Hop	n	L0	L1	L2	L3
1	10	0.337	0.206	0.337	0.352
2	104	0.234	0.281	0.215	0.251
3	15	0.123	0.131	0.111	**0.202**

Table 6: `gpt-4.1-nano` token-F1 by hop count across all four conditions. L3 takes the hop=3 slice from 0.131 (L1) to 0.202, a +0.071 absolute gain — the largest cell improvement in the table. The hop=1 L1 regression is the hard-mask artefact discussed in Section 6.3.

Scope	n	L0	L1	L2	L3
inter_year	53	0.220	0.276	0.222	0.264
intra	45	0.216	0.216	0.191	0.231
cross_company	24	0.264	0.315	0.221	0.275
fiscal_vs_calendar	5	0.301	0.222	0.301	0.280
forward_looking	2	0.138	0.157	0.138	0.138

Table 7: `gpt-4.1-nano` token-F1 by scope across all four conditions. L1 leads on *inter_year* and *cross_company* (matching the year-mask hypothesis); L3 recovers *intra* above L0/L1 and softens the *fiscal_vs_calendar* L1 regression (0.222→0.280) by following triples whose validity intersects the question's year set rather than masking entire chunks.

A3 tersification, A4 IDK-when-answerable, A5 parse error); five capture *corpus-level* limits (B1 fact absent, B2 forward-looking, B3 fiscal/calendar, B4 out-of-scope, B5 cross-filing); and NF marks non-failures (token-F1 $\geq$ 0.5 or correctly-abstaining "I don't know"). Stage 1 of the classifier resolves A3/A4/A5/B2/B3/B4/B5/NF deterministically using rule-based predicates in `src/taxonomy.py`; Stage 2 uses `gpt-4o-mini` with the prompt at `prompts/classify_failure_v1.txt` to disambiguate A1 vs. A2 vs. B1 on the remaining 1,183 rows. Headline counts are in Table 8; per-condition counts are in Table 9.

Code	Label	Count	%
A4	**IDK-when-answerable**	**1,508**	**41.8%**
A1	Retrieval miss	864	24.0%
NF	Non-failure	662	18.4%
A2	Hallucination	319	8.8%
B3	Fiscal/calendar	118	3.3%
A5	Parse error	60	1.7%
B5	Cross-filing	51	1.4%
B2	Forward-looking	25	0.7%
A3, B1, B4	(zero)	0	0.0%

Table 8: Headline failure-taxonomy counts across all conditions and models (N = 3,607). **A4 IDK-when-answerable dominates**: in 41.8 % of all predictions, the model abstains while the gold-bearing chunk is in its top-k.

A4 IDK-when-answerable is a generation ceiling, not a retrieval issue. Table 9 shows the per-condition counts. The A4 column moves only between 361 (L1) and 406 (L2): a 12 % spread across four *retrieval* conditions, while in the same comparison A1 (retrieval miss) varies from 193 to 268, a 39 % spread. Per-model the A4 spread is much larger: `gpt-4.1-nano` sits at 27 – 33 % A4 rate, while `gpt-oss-20B` and `gpt-oss-120B` sit at 48 – 54 % (Appendix D reproduces the per-(model, condition) matrix). The conclusion: A4 is driven by the answer model, not by the retriever. No retrieval intervention currently in scope can move the needle on it.

Cond.	A1	A2	A4	A5	B2/B3/B5	NF	total
L0	200	84	376	17	43	178	898
L1	268	62	361	13	60	139	903
L2	193	79	406	13	38	174	903
L3	203	94	365	17	53	171	903

Table 9: Failure-taxonomy counts per condition. A4 (IDK-when-answerable) moves only ±12 % across the four retrieval conditions, while A1 (retrieval miss) varies by ±39 %. This locates A4 in generation, not retrieval.

Why L2 regresses on this corpus. The taxonomy gives a clean explanation. Going from L0 to L2, A1 *drops* (200 → 193) — graph expansion does sometimes locate the gold-bearing chunk that pure cosine missed — but A2 hallucination *stays roughly flat* (84 → 79) and A4 IDK *rises* (376 → 406). The KG-expanded chunks are entity-related to the seeds but not necessarily directly relevant to the question; when handed to the answer model, they tend to dilute rather than clarify. Combined with the unchanged A4 ceiling, the net effect is a small F1 regression. L3 corrects half of this (A4 drops to 365) because temporal filtering re-tightens the entity walk and removes chunks that were only tangentially relevant.

Reliability. We sampled 20 Stage-2-ambiguous predictions stratified across the three Stage-2 outputs (A1, A2, B1) and re-classified them with `gpt-4o-2024-11-20` as a second LLM judge. Cohen's κ between the two judges is $\kappa = 0.200$ ("slight" on the Landis–Koch scale) at 60 % observed agreement. We frame this explicitly as *LLM-vs-LLM* consistency, *not* as human inter-rater reliability: a low κ here indicates that A1 (retrieval miss) versus A2 (hallucination) is a hard call when both judges face only the rendered prompt, and a stronger reliability claim would require human gold annotation we did not collect under the submission budget. A side-finding worth surfacing: across 1,183 Stage-2-ambiguous rows, gpt-4o-mini selected B1 (corpus-limit) zero times — it always preferred A1 (retrieval miss) or A2 (hallucination). The prompt allows B1 in principle; the model just appears reluctant to attribute failure to corpus structure when an entity-related chunk is in front of it. This is also captured in Appendix D.

6.2 Per-question difficulty

A complementary per-question view (the average F1 of every question across all 7 models × 4 conditions = 28 cells, computed by `scripts/build_question_difficulty.py`) sharpens two prior points. The 7-model-by-4-condition mean F1 across the 129 questions is 0.175, with median 0.156 and standard deviation 0.131. **Twenty questions (15.5 %) are universally failed** (F1=0 in every one of the 28 cells), while **zero questions reach a mean F1 of 0.8 across cells** — even our most consistently-solved question (`qid H019`, AAPL inter_year revenue) tops out at mean F1 = 0.719. This locates a hard upper bound on what the metric will ever read on this corpus, independently of any retrieval or generation improvement: SQuAD-style token-F1 *cannot* reach 1.0 when gold answers are verbose financial paragraphs and the model returns a concise number, even when the number is correct (the same mechanism behind the fiscal_vs_calendar regression in Section 5.3).

The easy and hard tails also tell a coherent ticker story. The ten easiest questions are *all* hop=2, with scope distribution inter_year 7 / cross_company 2 / intra 1, and ticker mass on AAPL (5) / AMZN (3) / MSFT (2). The ten hardest are mixed hop (hop=2 6 / hop=3 4) and lean intra (intra 6 / inter_year 4); the ticker mass shifts entirely to GOOGL / INTC / CSCO / ORCL / META (zero AAPL / AMZN / MSFT). One reasonable interpretation: AAPL, AMZN, and MSFT disclose financial line-items at a more machine-friendly granularity — their "Services revenue", "AWS operating income", etc., are named segments retrievers can match against verbatim, while GOOGL / INTC / CSCO use either coarser or more idiosyncratic segment vocabularies. We intentionally do not read this as "those companies are harder to QA"; it is a finding about the interaction of the issuer's disclosure style with the SQuAD-F1 metric.

6.3 Limitations

Five threats to validity. **(i)** The 129-item QA set is hand-vetted and internally consistent but small; the *forward_looking* cell ($n = 2$) and the *fiscal_vs_calendar* cell ($n = 5$) have too few items to support confident sub-claims on their own. A synthetic expansion pipeline (`scripts/synth_multihop_qa.py` + `verify_hop_count.py` + `auto_vet_synth.py`) produced 128 candidate items, of which 34 (26.6 %) passed the N-1-chunk hop verifier and only 4 (3.1 %) passed a stricter 4-axis LLM auto-vet (hop / answer / scope / leakage). We did *not* fold these 4 items into the reported evaluation because adding them would not move any CI by more than $\approx \pm 0.005$; the 129-item bank remains the basis for every result table. The synth pipeline itself is reported in Section 6.4 as a methodology lesson. **(ii)** Item 1A (Risk Factors) accounts for 35.4 % of fatal KG-extraction failures, because risk-factor prose is narrative rather than factual — the extraction prompt expects entity-relation triples that are simply not present. We accept this gap rather than

retune the prompt. **(iii)** The *fiscal_vs_calendar* L1 regression (-0.079, $n = 5$) is dominated by a token-F1 *tersification artefact* (Section 5.3): the L1 prediction is correct but shorter than gold, and SQuAD-style F1 penalises the brevity. This is a measurement issue, not a retrieval failure. **(iv)** All seven LLMs share a single answer prompt; we did not test prompt-level mitigations for A4 (e.g. "attempt an answer first, then optionally flag low confidence"). This is left as future work. **(v)** The corpus covers ten technology issuers across six fiscal years; findings should not be extrapolated to other sectors or to filings outside this window without further evaluation.

6.4 Lessons learned

The data refused to confirm three of our pre-registered expectations. We summarise what each refusal taught us, because each one is more useful for the next iteration than the design we wrote down at the start of the project.

We predicted KG would help; it regresses on its own. Our pre-registered hypothesis (H1, Section 3) said KG^{2}RAG would lift hop-$\geq$2 slices over L0. The 7-model average for L2 vs. L0 is $-6.7\,\%$. The taxonomy in Section 6.1 explains why: graph expansion does cut A1 retrieval miss ($200 \rightarrow 193$) but it also drags the model into A4 IDK more often ($376 \rightarrow 406$), because entity-related chunks are not all *question*-related. The lesson: an off-the-shelf graph walk needs an answer-side filter or a learned re-ranker; raw cosine on the expanded pool is not enough.

We predicted retrieval was the bottleneck; the A4 finding says generation is. The L0 vs. L1 `F1@answered` comparison (within ± 0.005 for every model) had already hinted that, given the right context, our generators were near a ceiling. The taxonomy then quantified it: $41.8\,\%$ of all predictions are A4 IDK-when-answerable, and the spread of A4 across retrieval conditions is $\pm 12\,\%$ while the spread across answer models is $30 - 53\,\%$. The lesson is methodological: when measuring a retrieval intervention, always report `F1@answered` alongside Token-F1, otherwise an abstention-happy model will look like a retrieval problem.

We predicted Wikipedia-derived multi-hop benchmarks would carry temporal signal; they do not. The decision to pivot away from HotpotQA / MuSiQue was the single highest-leverage choice in the project. Roughly half of the multi-hop "bridges" in those benchmarks are temporally static ("*who was born in X?*"), so a year mask cannot demonstrate lift even when the retriever is perfect. We had to construct a purpose-built corpus to make the temporal mechanism observable. The lesson: do the EDA before locking the benchmark; what looks like "multi-hop" in the literature can be temporally trivial.

We predicted Token-F1 would track correctness; in practice it has a ceiling. Across all $3{,}612$ cells, *no* question reaches F1 ≥ 0.8 in any condition. The bound is set by tersification (gold strings on this corpus are verbose; correct terse answers lose token overlap) plus the wrong-column extraction failure illustrated in Section 5.3 — L3 returns $\$9{,}201\,\mathrm{M}$ instead of $\$35{,}026\,\mathrm{M}$ and still scores 0.700 because every other token in the answer matches the gold frame. The lesson is that Token-F1 is necessary but not sufficient. A future iteration would supplement it with (i) entity-set extraction match against a gold entity slot and (ii) canonical-numeric-value match for financial metrics; together these would penalise the wrong-column failure mode that token overlap currently hides.

We predicted the synth verifier would accept 50% of generated questions; in the end, the strict pipeline accepted 3%. Our N-1-chunk hop verifier asks gpt-4o-mini to answer the synth question with each seed chunk dropped in turn; only items where every drop materially lowers F1 are kept. We expected $\approx 50\,\%$ pass on the basis of an early smoke-test. The full run produced $34/128 = 26.6\,\%$. A subsequent stricter 4-axis auto-vet (hop, answer, scope, leakage; `scripts/auto_vet_synth.py`) reduced the accept rate further to $4/128 = 3.1\,\%$, with the most common failure mode being `answer_correct` (94 / 128) — the synthesizer model frequently produces a plausible-looking answer that does not strictly match the seed chunks. There are two distinct lessons. **(a)** *Verify before targeting:* build the verifier first and let its observed pass-rate set the over-generation factor, not the other way around. **(b)** *Synthesizer models bias toward plausibility, not groundedness;* a future iteration would interleave the verifier into the synthesis loop (re-ask after each rejection) rather than running verification as a one-shot post-hoc filter.

Methodology that worked, and would carry to the next iteration. Three small process choices saved more time than they cost: (i) the two-stage rule + LLM classifier (rules absorb the easy 67 %, LLM only judges the ambiguous remainder, total cost $\approx$ \$0.30); (ii) caching every LLM output by rendered prompt, so re-runs after a prompt edit are free for the unchanged rows; (iii) dispatching implementation tasks to fresh subagents and letting them *stop and report BLOCKED* on plan bugs, which surfaced two real test/spec inconsistencies before we ran them at scale. We would re-use these patterns on a follow-up project without modification.

7 Conclusion and Future Work

We presented **TempoRAG-KG**, a 2×2 ablation that crosses a temporal year mask with a KG-augmented graph walk over a 25-filing, 7,467-chunk SEC 10-K corpus. Three findings were robust across all seven LLMs we tested. First, the temporal mask alone (L1) lifts token-F1 by a 7-model average of $+15.9\,\%$ over vanilla cosine retrieval (L0), with no model-level reversals. Second, the KG-augmented retriever alone (L2 KG^{2}RAG) regresses on this corpus by $-6.7\,\%$: graph expansion reaches more chunks but they are entity-related, not directly relevant, and dilute the cosine seeds. Third, the combined L3 condition recovers L1's overall lift and *wins specifically on hop=3* ($+0.071$ over L1 on `gpt-4.1-nano`) — the slice where the design predicts the KG will pay off. The dominant residual failure mode, accounting for 41.8 % of all predictions across all conditions, is *IDK-when-answerable*: the model refuses while the gold-bearing chunk is in its top-k. This relocates the next bottleneck from retrieval to generation.

Future work — inject-large, retrieve-small. A presentation reviewer pointed out that KG *injection* (i.e. extraction) and KG *retrieval* can use models of different capacity — a strong LLM for extraction (one-time, expensive but high-quality) and a small LLM for retrieval-time reasoning (cheap and many-shot). We did not test this split: `gpt-4.1-nano` served both roles in our pipeline. The cost decomposition in Table 4 suggests the upside is modest in absolute terms (KG extraction is already $<$ \$4 on our corpus) but the quality upside could be substantial: re-extracting Item 1A with `gpt-4o` or a recent reasoning model might recover much of the 35 % fatal rate we observe today and thereby raise the L2/L3 ceiling.

Future work — generation-side mitigations of A4. The A4 IDK rate is independent of retrieval condition but strongly correlated with answer-model identity (`gpt-oss-20B` sits at $\sim$ 50 %, `gpt-4.1-nano` at $\sim$ 30 %). A natural next experiment is a two-stage prompt: (i) the model is required to produce a candidate answer, (ii) a short verifier pass flags low confidence. We expect this to convert a substantial slice of A4 into either A2 (which we can detect with token-F1) or NF (correct answer), without changing the retriever at all.

Future work — expanded QA bank. Our 129-item QA set has 15 hop-3 questions; 34 verified hop-3+ synthetic questions are awaiting team vetting at the time of submission. Re-running the four conditions on the expanded pool would tighten the confidence intervals on the hop-3 cell where L3's advantage currently sits, and is the most direct way to confirm that the $+0.071$ headline reproduces.

References

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Bradley Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

Yao He, Ningyu Zhang, and Huajun Chen. 2024. Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Pranav Kishore and Collaborators. 2025. FinReflectKG-MultiHop: A multi-hop question answering benchmark over financial filings derived from a reflective knowledge graph. *Preprint*, arXiv:2510.02906.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Chenhan Xiong, Ali Payani, Francesco Barbieri, and Cornelia Caragea. 2025. TempAgent: Temporal-aware LLM agent for knowledge graph question answering. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics.

Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025. Knowledge graph-guided retrieval augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8912–8924. Association for Computational Linguistics.

A Extraction Prompt (v1)

See `prompts/extract_v1.txt` in the repository. A verbatim copy will be included in the final report.

B Answer Prompt

See `prompts/answer_v1.txt`. The prompt is deliberately minimal: it supplies the retrieved chunks, the question, and an instruction to refuse if the answer is not present in the context. Refusal phrasing is what the `is_idk` regex in `src/eval.py` is tuned to recognise.

C Scope Taxonomy

intra Answer is contained within a single (filing, item) pair.

inter_year Answer requires comparing two or more fiscal years of the same issuer.

cross_company Answer requires comparing two or more issuers, possibly in a single year.

fiscal_vs_calendar Question references a calendar-year event ("Q3 2022") that must be mapped onto the issuer's fiscal year.

forward_looking Answer is a projection or guidance statement rather than a realised value.

D Failure Taxonomy — per-(model, condition) A4 IDK rates

Table 10 reports the A4 IDK-when-answerable rate ($100 \times$A4 count/total predictions) per (model, condition) cell. The spread *across rows* (model identity) is much larger than the spread *across columns* (retrieval condition), supporting the Section 6.1 claim that A4 is generation-bound.

Model	L0	L1	L2	L3
gpt-4.1-nano	30.2%	27.9%	33.3%	27.1%
gpt-4o-mini	38.0%	35.7%	41.9%	35.7%
gpt-4o	41.1%	38.8%	44.2%	39.5%
llama-70b	41.9%	40.3%	46.5%	43.4%
llama-8b	41.1%	36.4%	41.9%	37.2%
gpt-oss-120B	50.4%	52.7%	53.5%	49.6%
gpt-oss-20B	50.4%	48.1%	53.5%	50.4%

Table 10: A4 IDK-when-answerable rate per (model, condition). Per-row range: 30 – 53 %. Per-column range: ±6 pp within each model. Generated by `scripts/build_a4_analysis.py`.

Reliability sample. Sample of 20 *Stage-2-ambiguous* predictions stratified across the three Stage-2 outputs (A1, A2, B1; B1 had zero rows in the source because Stage 2 never selects it). Each row was relabelled by `gpt-4o` as a second LLM judge using the same prompt as Stage 2. Cohen's κ between Stage 2 (`gpt-4o-mini`) and the judge (`gpt-4o`) is $\kappa = 0.200$ (Landis–Koch "slight"), at 60 % observed agreement. We restrict the sample to Stage-2-ambiguous rows on purpose: the judge prompt only allows A1/A2/B1 outputs, so sampling rule-labeled NF or A4 rows would compare apples-to-oranges. The judge's label distribution on the 20-row sample is 8 A1 + 12 A2; B1 was selected zero times by either judge. We explicitly do *not* claim this is a substitute for human inter-rater reliability. A low κ between two LLMs indicates that A1-vs-A2 is a difficult discrimination from the rendered prompt alone — the kind of judgement that benefits from an actual human adjudicator, which we did not collect under the submission budget. Audit trail: `data/eval/failure_taxonomy/kappa_llm_sample.jsonl` (both labels per row).

E Supplementary Tables

Table 11 extends Table 5 with full 95 % bootstrap CIs for every (model, condition) cell. The CIs are derived from $n = 1000$ resamples with a fixed seed.

Model	L0 (CI)	L1 (CI)	L2 (CI)	L3 (CI)
gpt-4.1-nano	0.229 [0.194, 0.270]	0.258 [0.221, 0.296]	0.212 [0.178, 0.250]	0.253 [0.217, 0.291]
gpt-4o-mini	0.230 [0.193, 0.274]	0.248 [0.206, 0.290]	0.214 [0.173, 0.258]	0.247 [0.206, 0.291]
gpt-4o	0.183 [0.141, 0.223]	0.221 [0.184, 0.264]	0.167 [0.129, 0.210]	0.194 [0.154, 0.234]
llama-3.3-70B	0.145 [0.112, 0.181]	0.179 [0.144, 0.215]	0.140 [0.105, 0.176]	0.157 [0.123, 0.195]
llama-3.1-8B	0.145 [0.114, 0.180]	0.180 [0.147, 0.216]	0.131 [0.101, 0.163]	0.153 [0.122, 0.186]
gpt-oss-120B	0.131 [0.094, 0.172]	0.138 [0.103, 0.181]	0.123 [0.088, 0.165]	0.127 [0.091, 0.165]
gpt-oss-20B	0.120 [0.088, 0.159]	0.145 [0.110, 0.184]	0.116 [0.082, 0.155]	0.118 [0.084, 0.156]

Table 11: Per-model token-F1 across all four conditions, with 95 % non-parametric bootstrap CIs ($n = 1000$, fixed seed). For every model the L1 mean is above the L0 mean and L3 is at least within the L0 CI; L2 falls below L0 in mean for every model.